T113 init 进程（pid 1）：异常向量、MMU 与地址翻译，t113_task、arm32_trapframe、内核栈、用户地址空间与 L1→L2 页表

t113_create_init_user_process() 之后、第一次 ret_from_fork 之前的状态；物理地址由页分配器动态给出，图里用 PA(…) 表示，数值来自源码常量

① 处理器模式与异常向量（ARMv7-A AArch32） · VBAR = kernel_vectors，8 个字，每项一条 b 指令

偏移	异常	进入模式 M[4:0]	该模式的分组寄存器	LR 修正 / 返回方式	miniOS 入口与处理
0x00	Reset	SVC 10011	—	不返回	kernel_reset：给 IRQ/ABT/UND/FIQ/SVC 设栈，写 VBAR，清 BSS
0x04	Undefined	UND 11011	SP_und LR_und SPSR_und	LR−4（ARM 态）	kernel_undefined_entry → t113_exception_report，停机
0x08	SVC	SVC 10011	SP_svc LR_svc SPSR_svc	LR = svc 下一条；movs pc, lr	kernel_svc_entry：压 68 B trapframe → t113_syscall_dispatch
0x0C	Prefetch Abort	ABT 10111	SP_abt LR_abt SPSR_abt	LR−4；读 IFSR / IFAR	kernel_prefetch_abort_entry → 报告 IFSR/IFAR，停机
0x10	Data Abort	ABT 10111	同上	LR−8；读 DFSR / DFAR	kernel_data_abort_entry → 报告 DFSR/DFAR，停机
0x14	未用	–	—	—	kernel_exception_hang：wfi 死循环（Hyp 模式的向量表才用此项）
0x18	IRQ	IRQ 10010	SP_irq LR_irq SPSR_irq	LR−4；srsdb … rfeia	kernel_irq_entry：srsdb sp!, #0x13 把 LR/SPSR 存到 SVC 栈 → cps #0x13 → t113_irq_handler
0x1C	FIQ	FIQ 10001	r8–r12 SP LR SPSR	LR−4	kernel_fiq_entry → 报告，停机

USR 10000 和 SYS 11111 共用 SP/LR；r0–r7、PC 所有模式共用。用户 SP/LR 只能用 ldm/stm 的 ^ 形式在特权模式下读写（kernel_svc_entry、arm32_return_to_user）。

模式栈：SVC 8 KiB，IRQ/ABT/UND/FIQ 各 1 KiB（链接脚本 .kernel_stack）；进程进内核后用自己的 4 KiB 内核栈作 SP_svc。

CPSR / SPSR（用户进程初值 0x00000010）

31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
N	Z	C	V	Q	IT	J	保留	GE	IT	E	A	I	F	T	M[4:0]
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	10000

M = 10000 USR；T = 0 ARM 态；I = F = 0 允许 IRQ/FIQ；A = 0。

movs pc, lr / rfeia 把 SPSR 拷回 CPSR，完成模式切换。

与 AArch64 的根本区别（对照图 1.2）

- 用 PL0/PL1 + 模式（miniOS 用 7 种），异常模式各有 SP/LR/SPSR
- 向量表 8 项 × 4 B（一条指令），不是 16 项 × 0x80 B
- 系统寄存器在 CP15 里，用 MCR/MRC 访问，不是 MSR/MRS
- 返回靠 movs pc / subs pc / rfe，不是 eret + ELR

miniOS 用到的 CP15 寄存器（MCR/MRC p15, op1, Rt, CRn, CRm, op2）

SCTLR	c1,c0,0	M=1 I=1 XP=1 · C=0 TRE=0 AFE=0
TTBR0 / TTBCR	c2,c0,0 / c2,c0,2	内核 L1 或 mm->l1 / N = 0
DACR	c3,c0,0	0x1：domain 0 = client
DFSR/IFSR · DFAR/IFAR	c5,c0,0/1 · c6,c0,0/2	故障状态与故障地址
ATS1C{P,U}{R,W} → PAR	c7,c8,0..3 → c7,c4,0	硬件代查一次翻译，不触发异常
TLBIALL · ICIALLU · DCCIMVAC	c8,c7,0 · c7,c5,0 · c7,c14,1	换 TTBR0 全刷 TLB；装代码后同步 I/D
VBAR	c12,c0,0	kernel_vectors

② ARMv7-A 短描述符格式（TTBCR.EAE = 0） · 32 位 VA，两级：L1 4096 项 × 1 MiB，L2 256 项 × 4 KiB

L1 段描述符（内核 1:1 映射用） 位[1:0] = 10

31:20	19	18	17	16	15	14:12	11:10	9	8:5	4	3	2	1:0
段基址 PA[31:20]	NS	0	nG	S	AP2	TEX	AP		Domain	XN	C	B	类型
VA[31:20]	0	0	0	D:1	0	N:001	01	0	0	D:1	N:1	1	10

DRAM：0x4020140e 外设：0x02510416

N = Normal（SRAM/DRAM），D = Device（外设）；AP=01 仅特权读写

L1 页表描述符（用户 L1[1]、L1[8]） 位[1:0] = 01

31:10	9	8:5	4	3	2	1:0
L2 表基址 PA[31:10]（1 KiB 对齐）		Domain	0	NS	PXN	类型
PA(l2a) / PA(l2b)	0	0	0	0	0	01

PA(l2b) | 0x01

get_l2()：l2 & 0xfffffc00 | L1_COARSE

L2 小页描述符（用户页） 位[1] = 1

31:12	11	10	9	8:6	5:4	3	2	1	0
页基址 PA[31:12]	nG	S	AP2	TEX	AP	C	B	1	XN
PA(code) / PA(ustack)	1	1	0	001	10/11	1	1	1	0/1

代码 0xc6e 栈 0xc7f

l2_flags()：AP 10 用户只读 / 11 用户读写；数据页 XN=1

没用到的格式：

- 超级段（16 MiB，L1 位18 = 1）
- L2 大页（64 KiB，类型 01）
- LPAE 长描述符（TTBCR.EAE = 1，三级、64 位项）

Cortex-A7 支持，miniOS 不用

③ 两次真实的翻译：init 的第一次取指和第一次写栈（来自 _init 反汇编，e_entry = 0x00100000）

```
0x00100000  eb000002  bl main          →  用户态取指 VA 0x00100000
0x00100010  e92d4030  push {r4, r5, lr} →  SP 0x00900000 先减 12，写 0x008ffff4 / f8 / fc（STMDB）
```

VA 0x00100000 [31:20]=0x001 [19:12]=0x00 [11:0]=0x000

→ L1 描述符地址 = TTBR0[31:14] : 0x001 : 00

→ **L1[1] = PA(l2a) | 0x01**

位[1:0] = 01 → 去 L2；Domain 0 client → 查 AP

→ L2 描述符地址 = L1desc[31:10] : 0x00 : 00

→ **L2[0] = PA(code) | 0xc6e**

AP=10 用户只读，XN=0 → 取指通过

→ PA = L2desc[31:12] : VA[11:0]

→ **PA(code) + 0x000**

结果进 TLB；下次同页不再走表

VA 0x008ffff4 [31:20]=0x008 [19:12]=0xff [11:0]=0xff4

→ L1 描述符地址 = TTBR0[31:14] : 0x008 : 00

→ **L1[8] = PA(l2b) | 0x01**

位[1:0] = 01 → 去 L2；Domain 0 client → 查 AP

→ L2 描述符地址 = L1desc[31:10] : 0xff : 00

→ **L2[255] = PA(ustack) | 0xc7f**

AP=11 用户读写 → 写通过；XN=1，此页不能取指

→ PA = L2desc[31:12] : VA[11:0]

→ **PA(ustack) + 0xff4**

结果进 TLB；下次同页不再走表

内核地址只查一级：VA 0x40200000 → L1[0x402] = 0x4020140e 段 → PA 0x40200000；VA 0x02500c00（UART3）→ L1[0x025] = 0x02510416 → PA 0x02500c00。

表遍历本身按 TTBR0 的 RGN = 0（不经 cache）读内存；SCTLR.C = 0 时数据访问也不经 D-cache，描述符里的 C/B 位暂时只是“将来可缓存”的声明。

走不通时：FS = DFSR/IFSR {[10], [3:0]}

0b00101	段翻译错：L1 项类型 00
0b00111	页翻译错：L2 项为 0
0b01101	段权限错：用户碰内核段
0b01111	页权限错：用户写只读页
0b01001/01011	域错（DACR 为 no access）

读写 → 0x10 Data Abort（DFAR = VA）

取指 → 0x0C Prefetch Abort（IFAR = VA）

syscall 检查用户指针：ATS1CUR/CUW 查 PAR.F

t113_task · PA(task)，独占一页（4 KiB）

cpu_context（第一个字段，偏移 0：switch.S 把 task 指针当作 context 指针） +0

r4	PA(task)	ret_from_fork 里 mov r0, r4 → t113_finish_fork(task)
r5 / r6	0 / 0	内核线程才用：fn / arg（用户进程为 0）
r7..r11	0	其余被调用者保存寄存器
sp	PA(stack)+0xfbc	指向内核栈顶的 trapframe ——→
lr	ret_from_fork	cpu_switch_to 末尾 bx lr 跳过去

共 10 个字（40 B）：ldmia r1, {r4-r11, sp, lr} 一条指令装回

pid/state	1 / RUNNING	flags = PF_USER，name = "init-user"
kernel_stack	PA(stack)	另一页；与 task 页不一定相邻
trapframe	PA(stack)+0xfbc	= kernel_stack + 4096 − 68
mm	PA(vm)	struct t113_user_vm：l1、l2[8]、pages[32]、brk
files[4]	…	就绪环、任务链表、parent、vfork_parent 等

内核栈页 [PA(stack), PA(stack)+0x1000) · SVC 模式的 sp

PA(stack)+0x1000 栈顶

struct arm32_trapframe（68 B = 0x44）

r[0..12]	52 B	+0..+48 全 0
user_sp	+52	0x00900000 → SP_usr
user_lr	+56	0 → LR_usr
pc	+60	ELF e_entry → LR_svc → PC
cpsr	+64	0x10 → SPSR_svc → CPSR

TF_USER_SP/TF_PC/TF_CPSR 由 _Static_assert 和 fork_entry.S 钉死

PA(stack)+0xfbc = 栈顶 − 0x44

↓ 空闲内核栈（向低地址增长，约 4 KiB − 68 B）

之后每次 svc：kernel_svc_entry 在当前 sp 下方再压一个 68 B 帧，返回时弹掉，所以 fork 复制的是 parent_tf，不是 task->trapframe；

被切走：cpu_context.sp 存成比 +0xfbc 更低的值。

PA(stack) 栈底

movs pc, lr 时的寄存器（arm32_return_to_user）

TTBR0	PA(l1)	用户 L1 表（16 KiB，每进程一份）
TTBR1	不用	TTBCR.N = 0：TTBR0 管整个 4 GiB
DACR	0x1	domain 0 = client，按 AP 位检查
SP_usr	0x00900000	ldmia r1, {sp, lr}^
LR_usr	0x0	同一条指令写入
LR_svc	e_entry	ldr lr, [r3, #TF_PC]
SPSR_svc	0x10	msr spsr_cxsf；USR 模式，I/F 打开

movs pc, lr：PC ← LR_svc，CPSR ← SPSR_svc，进入用户态

和 AArch64 版对照：

eret ← ELR_EL1 / SPSR_EL1，SP_EL0 ← pt_regs.sp

ARMv7 没有 ELR，用 SVC 模式的 LR 和 SPSR；

用户 SP/LR 是分组寄存器，只能用 ^ 形式的 ldm/stm 访问。

0x10 = USR，I=0、F=0：用户态可以被定时器打断。

TTBR0 = mm->l1 = PA(l1)（遍历起点）

PA(task) — t113_task — 含 cpu_context

PA(stack) — 内核栈 — 顶部 68 B 是 trapframe

PA(vm) — t113_user_vm — task->mm，一页

PA(l1) · 用户 L1 表（16 KiB，4096 项 × 1 MiB）

先整份复制 __kernel_l1_table，再改两项：

[1] = PA(l2a) | 0x1 coarse → L2 表 A

[8] = PA(l2b) | 0x1 coarse → L2 表 B

[0,2..7,9..15] SRAM 段，仅特权 RW

[0x010..0x3ff] 外设段，Device，XN

[0x400..0xbff] DRAM 段，1:1，仅特权 RW

其余为 0（访问即 abort）

PA(l2a) · L2 表 A（VA 0x001xxxxx）

[0] = PA(code) | 0xc6e

[1..] 代码/数据段的后续页

PA(l2b) · L2 表 B（VA 0x008xxxxx）

[255] = PA(ustack) | 0xc7f

L2 表 256 项 × 4 B = 1 KiB，占一整页

PA(code) — 物理页：用户代码 — 用户只读、可执行

PA(ustack) — 物理页：用户栈 — 用户读写、XN

其它页 — 堆（sbrk 按页映射）；ELF 的其余段；每进程最多 32 页、8 张 L2

物理页（地址从左到右没有顺序）：箭头就是 MMU 的遍历，只有两级——L1 项给出 L2 表，L2 项给出 4 KiB 页

L1 表要 16 KiB 对齐，所以用 t113_alloc_pages(4, 4) 分配；L2 表只有 1 KiB，却各占一页

AArch64 版是 4 级（L0→L3），每级一页 512 项；ARMv7 短描述符只有 2 级

用户虚拟地址空间（TTBR0 = PA(l1)；只有用户页不是 1:1 映射）

0x00900000 ← SP_usr

用户栈（RW）

0x008ff000

VA [0x008ff000, 0x00900000)

USER_STACK_VA，一页，读写、XN

L1[8] → L2B[255] → PA(ustack)

映射到

PA(ustack) — 物理页：用户栈 — L2 描述符低 12 位 0xc7f

堆：从 heap_base（ELF 末尾向上取整）开始，sbrk 按页映射

其余未映射：L1 项是内核 SRAM 段（仅特权）或 0

用户访问 → Permission / Translation fault

未映射 / 堆

0x001xxxxx

用户映像（代码）

0x00100000

VA [0x00100000, …) USER_CODE_VA

ELF PT_LOAD 段，按 p_flags 决定 RW / X；e_entry 在这里

L1[1] → L2A[0] → PA(code)

映射到

PA(code) — 物理页：用户代码 — L2 描述符低 12 位 0xc6e

[0, 0x00100000)：L1[0] 是内核 SRAM 段，用户无权访问

0x0

用户栈 VA = 0x008ff000

31:20	19:12	11:0
L1 = 8	L2 = 255	偏移 = 0
L1 索引	L2 索引	页内偏移

代码 VA = 0x00100000

31:20	19:12	11:0
L1 = 1	L2 = 0	偏移 = 0
L1 索引	L2 索引	页内偏移

这些下标就是上面页表框里的 [8] [255] 和 [1] [0]。

AArch64 版把 48 位 VA 切成 9+9+9+9+12；ARMv7 的 32 位 VA 只切 12+8+12。

每进程 L1 都是内核 L1 的完整副本，所以内核代码在用户页表下照样可用，

代价是 fork/exec 都要复制 16 KiB，切换时 TLBIALL 全刷。

L2 小页描述符解码（SCTLR.XP = 1，TRE = 0，AFE = 0）

字段	位	代码页 0xc6e	栈页 0xc7f	含义
XN	0	0 可执行	1 不可执行	数据页一律 XN
类型	1	1 小页	1 小页	4 KiB
B / C	2 / 3	1 / 1	1 / 1	写回、写分配
AP[1:0]	5:4	10 用户只读	11 用户读写	特权总是可读写
TEX	8:6	001	001	Normal 内存
AP[2]	9	0	0	不加只读限制
S	10	1	1	可共享
nG	11	1	1	非全局；未用 ASID，整表刷

- L1 coarse 描述符：PA(l2) & 0xfffffc00 | 0x1（domain 0）。内核 1 MiB 段：AP = 01，仅特权读写。
- 代码页对内核仍可写：exec 复制段内容后，sync_executable_pages() 用 DCCIMVAC + ICIALLU 同步。

这几样东西的关系（和 AArch64 版逐条对应）

① **alloc_process_common()：分配 task 页和内核栈页，trapframe 放在栈顶下 68 B；**

cpu_context.r4 = task，sp = trapframe，lr = ret_from_fork（AArch64：x19 = fn，pc = ret_from_fork）。

② **t113_user_vm_create_elf()：复制内核 L1，按 PT_LOAD 建 L2 和页，再映射用户栈页；**

trapframe.pc = e_entry，user_sp = 0x00900000，cpsr = 0x10。

③ **第一次调度：t113_cpu_switch_to 装回 r4-r11、sp、lr，bx lr → ret_from_fork；**

t113_finish_fork(task) 写 TTBR0 = mm->l1（AArch64 在 move_to_user_mode 里写 TTBR0_EL1）。

④ **arm32_return_to_user：{sp, lr}^ 写用户分组寄存器，SPSR_svc = cpsr，**

ldmia r0-r12，movs pc, lr 进入 USR 模式（AArch64：kernel_exit + eret）。

⑤ **之后每次 svc：kernel_svc_entry 在 SVC 栈上新压一个 68 B 帧，不复用 task->trapframe；**

AArch64 版的 pt_regs（272 B）则总在栈顶同一位置。

⑥ **内核地址：T113 内核在 DRAM 1:1 映射，用户 L1 里也有这些段（仅特权）；**

AArch64 版内核走 TTBR1（高地址），用户页表里没有内核映射。